

#9 Building REST APIs

Building REST APIs	Controllers, request mapping, path variables, query parameters, request bodies, response entities, DTOs, API design principles
--------------------	--

Building REST APIs

REST API (Representational State Transfer API) is a way for applications to communicate over HTTP using standard methods like GET, POST, PUT, DELETE.

Example

Think of a food delivery app:

Action	HTTP Method	API
View all restaurants	GET	<code>/restaurants</code>
View one restaurant	GET	<code>/restaurants/1</code>
Add restaurant	POST	<code>/restaurants</code>
Update restaurant	PUT	<code>/restaurants/1</code>
Delete restaurant	DELETE	<code>/restaurants/1</code>

Controller

A Controller receives HTTP requests and returns responses.

Why do we use it?

Without a controller, Spring Boot doesn't know which method should handle incoming requests.

Example

```
@RestController
public class StudentController {

    @GetMapping("/hello")
```

```
public String hello() {  
    return "Hello Student";  
}
```

Annotations

@RestController

Combines:

@Controller

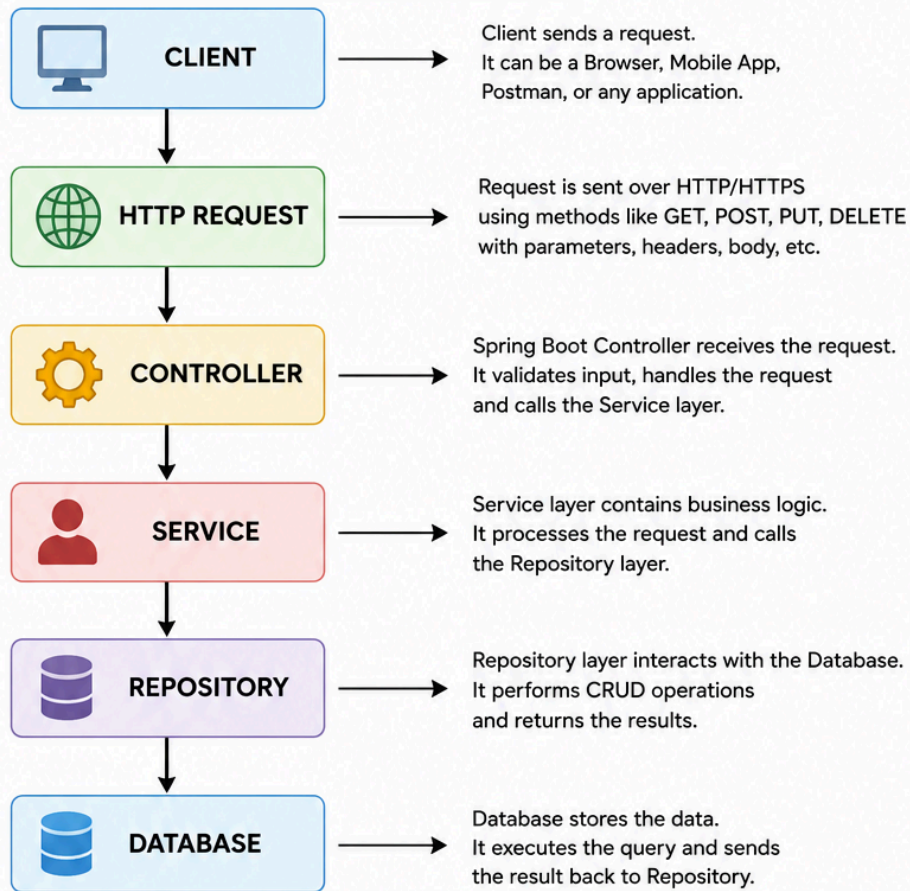
@ResponseBody

Meaning:

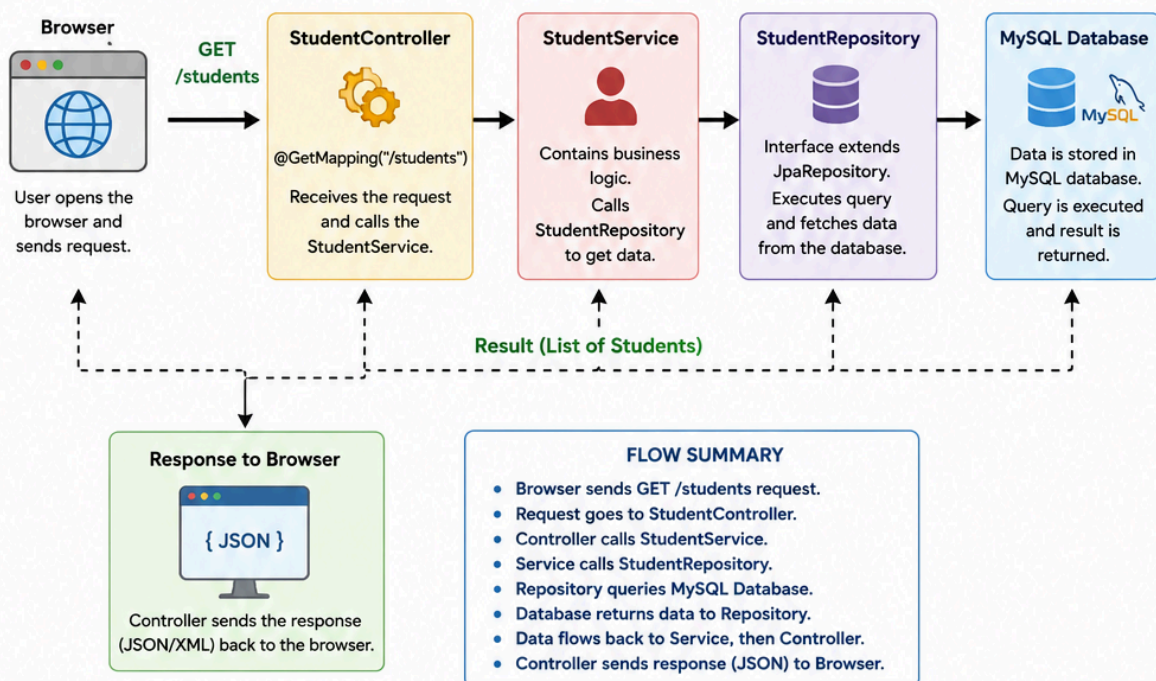
- Handle requests
- Return JSON/String directly

byteXL

REQUEST FLOW



EXAMPLE: GET /STUDENTS



2. Request Mapping

Used to map URLs with methods.

@RequestMapping

```
@RestController
@RequestMapping("/students")
public class StudentController {

}
```

Now every endpoint starts with:

/students

Example

```
@GetMapping
public String getAllStudents() {
    return "All Students";
}
```

URL:

GET /students

Specialized Mapping Annotations

GET

@GetMapping("/students")

Used to fetch data.

Example

```
@GetMapping("/students")
public List<Student> getStudents() {
    return service.getStudents();
}
```

POST

Used to create data.

`@PostMapping("/students")`

Example:

```
@PostMapping("/students")
public Student addStudent(@RequestBody Student student) {
    return service.save(student);
}
```

PUT

Used to update complete object.

`@PutMapping("/students/{id}")`

Example

```
@PutMapping("/students/{id}")
public Student updateStudent(
    @PathVariable Long id,
    @RequestBody Student student) {

    return service.update(id, student);
}
```

DELETE

`@DeleteMapping("/students/{id}")`

Example

```
@DeleteMapping("/{students}/{id}")
public String deleteStudent(@PathVariable Long id) {
    service.delete(id);
    return "Deleted";
}
```

Path Variables

Path Variable means value comes from URL path.

Example URL

/students/101

Here:

101 is path variable.

Code

```
@GetMapping("/{students}/{id}")
public String getStudent(
    @PathVariable Long id) {

    return "Student ID: " + id;
}
```

Request

GET /students/101

Response

Student ID: 101

Multiple Path Variables

```
@GetMapping("/students/{id}/courses/{courseId}")  
public String getCourse(  
    @PathVariable Long id,  
    @PathVariable Long courseId) {  
  
    return id + ": " + courseId;  
}
```

Request

GET /students/1/courses/10

Response

1: 10

Query Parameters

Extra information sent after ?

URL

/students?name=Ummed

Code

```
@GetMapping("/students")  
public String searchStudent(  
    @RequestParam String name) {  
  
    return name;  
}
```

Request

GET /students?name=Ummed

Response

Ummed

Multiple Query Parameters

```
@GetMapping("/students")
public String getStudent(
    @RequestParam String name,
    @RequestParam int age){

    return name + " " + age;
}
```

Request

GET /students?name=Ummed&age=25

Optional Query Parameter

```
@GetMapping("/students")
public String getStudent(
    @RequestParam(required=false)
    String name){

    return name;
}
```

Path Variable vs Query Parameter

Path Variable	Query Parameter
Mandatory Resource	Optional Filtering
/students/1	/students?name=ummed
Identifies specific object	Filters/Searches data

Request Body

Used when sending complex data.

JSON Request

```
{  
  "name": "UmmedI",  
  "email": "ummedsingh@gmail.com",  
  "age": 25  
}
```

Java Class

```
public class Student {  
  
    private String name;  
    private String email;  
    private int age;  
  
    // getters setters  
}
```

Controller

```
@PostMapping("/students")  
public Student addStudent(  
    @RequestBody Student student){  
  
    return student;  
}
```

Spring automatically converts:

JSON to Student Object

using Jackson Library.

ResponseEntity

Used to control:

- Status Code
- Headers
- Response Body

Instead of:

```
return student;
```

Use:

```
return ResponseEntity.ok(student);
```

Example

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudent(
    @PathVariable Long id){

    Student student = service.find(id);

    return ResponseEntity.ok(student);
}
```

Response: 200 OK

Created Response

```
@PostMapping
public ResponseEntity<Student> save(
    @RequestBody Student student){

    Student saved = service.save(student);

    return ResponseEntity.status(HttpStatus.CREATED)
```

```
        .body(saved);  
    }
```

Response: 201 CREATED

Not Found Response

```
@GetMapping("/{id}")  
public ResponseEntity<?> getStudent(  
    @PathVariable Long id){  
  
    Student student = service.find(id);  
  
    if(student == null){  
        return ResponseEntity.notFound().build();  
    }  
  
    return ResponseEntity.ok(student);  
}
```

Response: 404 NOT FOUND

DTO (Data Transfer Object)

One of the most important concepts.

Problem

Suppose Entity:

```
@Entity  
public class User {  
  
    private Long id;  
    private String name;  
    private String email;
```

```
private String password;  
}
```

If returned directly:

```
return user;
```

Response:

```
{  
  "id":1,  
  "name":"Ummed",  
  "email":"ummedsingh@gmail.com",  
  "password":"12345"  
}
```

Password exposed

Solution: DTO

```
public class UserDTO {  
  
    private Long id;  
    private String name;  
    private String email;  
}
```

Controller returns:

UserDTO

Response:

```
{  
  "id":1,  
  "name":"Ummed",  
  "email":"ummedsingh@gmail.com"  
}
```

Password hidden

Benefits of DTO

- **Security:** Hide sensitive data.
- **Performance:** Send only required fields.
- **Flexibility:** API structure independent of database.
- **Maintainability:** Database changes don't affect clients.

DTO Mapping Example

Entity

```
User user = repository.findById(id);
```

DTO

```
UserDTO dto = new UserDTO();  
  
dto.setId(user.getId());  
dto.setName(user.getName());  
dto.setEmail(user.getEmail());
```

Return DTO:

```
return dto;
```

API Design Principles

1. Use Nouns, Not Verbs

Bad

- /getStudents
- /createStudent

- /deleteStudent

Good

- /students
- /students/1

HTTP methods define action.

2. Use Proper HTTP Methods

- GET Read
- POST Create
- PUT Update
- DELETE Delete

3. Use Plural Names

- **Bad:** /student
- **Good:** /students

4. Use Correct Status Codes

Code	Meaning
200	Success
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error

5. Version APIs

- /api/v1/students
- /api/v2/students

6. Use Pagination

Bad

- GET /students
- returns 1 million records.

Good

- GET /students?page=0&size=10

7. Consistent Response Structure

Good Practice:

```
{
  "success": true,
  "message": "Student Found",
  "data": {
    "id": 1,
    "name": "Ummed"
  }
}
```

byteXL

REST API Example

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping
    public ResponseEntity<List<StudentDTO>> getAllStudents() {
        return ResponseEntity.ok(service.getAllStudents());
    }
}
```

```
@GetMapping("/{id}")
public ResponseEntity<StudentDTO> getStudent(
    @PathVariable Long id) {

    return ResponseEntity.ok(
        service.getStudent(id));
}

@PostMapping
public ResponseEntity<StudentDTO> saveStudent(
    @RequestBody StudentDTO dto) {

    StudentDTO saved =
        service.saveStudent(dto);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(saved);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteStudent(
    @PathVariable Long id) {

    service.delete(id);

    return ResponseEntity.noContent()
        .build();
}
}
```

Interview Questions

Difference between @PathVariable and @RequestParam?

- PathVariable identifies a resource.
- RequestParam filters/searches data.

Why use ResponseEntity?

- Control status code
- Control headers
- Control response body

Difference between PUT and POST?

- POST creates new resource.
- PUT updates existing resource.

What is REST?

An architectural style that uses HTTP methods and resource-based URLs for communication between client and server.

Quick Revision Sheet

@RestController → REST API Controller

@RequestMapping → Base URL

@GetMapping → Read

@PostMapping → Create

@PutMapping → Update

@DeleteMapping → Delete

@PathVariable → Value from URL

@RequestParam → Value from Query String

@RequestBody → JSON to Java Object

ResponseEntity → Control Response

DTO → Data Transfer Object

200 → Success

201 → Created

404 → Not Found

500 → Server Error